

SKRIPSI

**OPTIMASI SISTEM HMI SUMBER TERBUKA PADA PLC OMRON
BERBASIS PROTOKOL FINS PADA PT XYZ**

Disusun dan diajukan oleh



**PROGRAM STUDI COMPUTER SCIENCE STUDY PROGRAM
DEPARTEMEN
FAKULTAS BINUS ONLINE LEARNING
UNIVERSITAS BINA NUSANTARA
JAKARTA**

2025

**OPTIMASI SISTEM HMI SUMBER TERBUKA PADA PLC OMRON
BERBASIS PROTOKOL FINS PADA PT XYZ**

SKRIPSI

Diajukan sebagai salah satu syarat untuk memperoleh gelar pada Program Studi
Computer Science Study Program Departemen Fakultas BINUS Online Learning
Universitas Bina Nusantara Jakarta

**PROGRAM STUDI COMPUTER SCIENCE STUDY PROGRAM
DEPARTEMEN
FAKULTAS BINUS ONLINE LEARNING
UNIVERSITAS BINA NUSANTARA
JAKARTA**

2025

HALAMAN PERNYATAAN KEOTENTIKAN

Yang bertanda tangan di bawah ini:

Nama :

NIM :

Program Studi : Computer Science Study Program

Jenjang : S1

Menyatakan dengan ini bahwa karya tulisan saya berjudul

OPTIMASI SISTEM HMI SUMBER TERBUKA PADA PLC OMRON BERBASIS PROTOKOL FINS PADA PT XYZ

Adalah benar hasil karya saya sendiri bukan merupakan pengambilan alihan tulisan orang lain dan belum pernah dipublikasikan dalam bentuk apapun.

Apabila dikemudian hari terbukti atau dapat dibuktikan bahwa sebagian atau keseluruhan skripsi ini merupakan hasil karya orang lain, maka saya bersedia menerima sanksi atas perbuatan tersebut.

Jakarta, 2025

NIM.

OPTIMASI SISTEM HMI SUMBER TERBUKA PADA PLC OMRON BERBASIS PROTOKOL FINS PADA PT XYZ

Disusun dan diajukan oleh

Telah dipertahankan di hadapan Panitia Ujian yang dibentuk dalam rangka
Penyelesaian Studi Program Sarjana pada Program Studi Computer Science Study
Program Fakultas BINUS Online Learning Universitas Bina Nusantara dan
dinyatakan telah memenuhi syarat kelulusan

Menyetujui,

Pembimbing Utama

Pembimbing Pertama

NIP.

NIP.

Ketua Program Studi

NIP.

HALAMAN PENGESAHAN

Skripsi ini diajukan oleh:

Nama :
NIM :
Program Studi : Computer Science Study Program
Judul Skripsi : OPTIMASI SISTEM HMI SUMBER TERBUKA PADA PLC
OMRON BERBASIS PROTOKOL FINS PADA PT XYZ

Telah dipertahankan di depan Tim Penguji dan diterima sebagai bagian persyaratan untuk memperoleh gelar pada Program Studi Computer Science Study Program Fakultas BINUS Online Learning Universitas Bina Nusantara.

DEWAN PENGUJI

Tanda Tangan

1. Ketua : (.....)
2. Sekretaris : (.....)
3. Anggota : (.....)
4. Anggota : (.....)

Ditetapkan di : Jakarta

Tanggal : 2025

KATA PENGANTAR

Puji syukur penulis panjatkan kepada Allah SWT, karena atas berkat dan rahmat-Nya penulis dapat menyelesaikan skripsi ini. Shalawat serta salam senantiasa tercurah kepada *Rasulullah* Muhammad *Shallallahu Alaihi Wasallam*, yang merupakan teladan dalam menjalankan kehidupan dunia.

Alhamdulillah, skripsi dengan judul "IMPLEMENTASI FILTER SPASIAL LINEAR PADA VIDEO *STREAM* MENGGUNAKAN *FPGA HARDWARE ACCELERATOR*" yang disusun sebagai salah satu syarat untuk mencapai gelar Sarjana pada program studi Sistem Informasi fakultas Matematika dan Ilmu Pengetahuan Alam Universitas Hasanuddin ini dapat diselesaikan. Walaupun adanya kendala-kendala yang dihadapi khususnya wabah Covid-19 ketika skripsi ini dikerjakan. Tetapi dalam penulisan skripsi ini, penulis mampu menyelesaikan pada waktu yang tepat berkat bantuan dan dukungan dari berbagai pihak.

Ucapan terima kasih dan apresiasi yang tak terhingga kepada kedua orang tua penulis bapak **Sudarmin** dan ibu **Yuli Hadiyanti** yang tak kenal lelah dalam memanjatkan doa serta memberikan nasihat dan motivasi kepada penulis. Tak lupa juga kepada saudara-saudara penulis **Fitri Handayani, Tri Novianti, Jumadil Yusuf, Muhammad Fitrah, Adam Ramadhan** yang selalu menjadi motivasi bagi penulis untuk terus melangkah maju.

Penulis menyadari bahwa skripsi ini dapat terselesaikan dengan adanya bantuan, bimbingan, dukungan dan motivasi dari berbagai pihak. Oleh karena itu, penulis mengucapkan ucapan terima kasih dengan tulus kepada:

1. Rektor Universitas Hasanuddin, Ibu **Prof. Dr. Dwia Aries Tina Pulubuhu** beserta jajarannya.
2. Dekan Fakultas Matematika dan Ilmu Pengetahuan Alam, **Dr. Eng. Amiruddin** beserta jajarannya.
3. Ketua Departemen Matematika FMIPA, **Dr. Nurdin, S.Si., M.Si**, dan juga **Dr. Muhammad Hasbi, M.Sc** sebagai ketua Program Studi Sistem Informasi Universitas Hasanuddin.
4. Bapak **Dr. Eng. Armin Lawi, S.Si., M.Eng** sebagai pembimbing utama yang

telah banyak memberikan arahan, ide, motivasi serta dukungan kepada penulis dalam banyak hal.

5. Almarhum bapak **Dr. Diaraya, M.Ak** dan bapak **Supri Bin Hj. Amir, S.Si., M.Eng** sebagai pembimbing pertama yang senantiasa memberikan masukan kepada penulis.
6. Bapak **Dr. Hendra, S.Si., M.Kom** dan Ibu **Nur Hilal, S.Si., M.Si** sebagai tim penguji atas saran dan masukan pada penelitian yang telah dilakukan oleh penulis.
7. Seluruh Bapak dan Ibu dosen FMIPA Universitas Hasanuddin yang telah mendidik dan memberikan ilmunya sehingga penulis mampu menyelesaikan program sarjana. Serta para staf yang telah membantu dalam pengurusan berkas administrasi.
8. Saudara-saudara **Ramsis Squad (Sultan, Muh Rizaldi, Sangereng Dewa Raja, Hajrin, Badaruddin Hidayat, Hamzah Julianto Nugraha, Muh Naim, Achmad Husein Nyompa)** sebagai keluarga semasa tinggal di Ramsis sejak menjadi mahasiswa baru, saling berbagi dalam banyak hal, saling membantu dan bahkan saling merepotkan.
9. Saudara-saudara **Sunu Squad** dan **SSC Squad (Akbar, Muh Fikri Satria A, Andi Rezki Muh Nur, Muhammad Akbar Atori, Baharuddin Kasim, Andi Yaumil Falakh, Nur Ikhwan Putra Pratama, Bagas Prasetyo, Zinedine Kahlil Gibran Zidane, Rio Mukhtarom, Marfiandhi Putra, Abdul Aziz Mubarak, Mutawally Syarawy, Fatur Rahman, Fitriadi Syawal Mustafa)** yang telah menemani penulis selama perkuliahan, saling memberi motivasi dan bantuan, meluangkan waktu dan berbagi suka-duka serta kebersamaan selama menuntut ilmu.
10. Saudari **Suci Rahmadana Anwar** dan **Sri Juliana** yang senantiasa menemani, memberi nasihat, menjadi tempat bertanya, serta dukungan untuk menyelesaikan skripsi ini.
11. Keluarga besar **Ilmu Komputer Unhas 2016** yang setia menemani dan membantu penulis selama menjalani pendidikan. Serta kakak-kakak dan adik-adik **Ilmu Komputer 2014, 2015, 2017, 2018** yang telah banyak

membantu, semoga tetap semangat dalam mengejar impian.

12. Keluarga besar **HIPERMAWA Koperti Unhas** yang senantiasa memberikan naungan kekeluargaan dan dukungan.
13. Rekan-rekan **KKN Internasional Jepang Unhas Gel. 102** yang telah menjadi keluarga baru selama KKN dan menjadikan KKN sebagai momen yang berkesan.
14. Serta semua pihak yang telah banyak berpartisipasi, baik secara langsung maupun tidak langsung dalam penyusunan skripsi ini yang tidak sempat penulis sebutkan satu per satu.

Penulis menyadari bahwa skripsi ini masih jauh dari sempurna dikarenakan terbatasnya pengalaman dan pengetahuan yang dimiliki penulis. Oleh karena itu, penulis mengharapkan segala bentuk saran serta masukan bahkan kritik yang membangun dari berbagai pihak. Semoga tulisan ini memberikan manfaat kepada semua pihak yang membutuhkan dan terutama untuk penulis.

Makassar, 9 April 2021

DENNY CHRISNANDA

NIM. H13116002

PERNYATAAN PERSETUJUAN PUBLIKASI TUGAS AKHIR UNTUK KEPENTINGAN AKADEMIS

Sebagai civitas akademik Universitas Bina Nusantara, saya yang bertanda tangan di bawah ini:

Nama :
NIM :
Program Studi : Computer Science Study Program
Departemen :
Fakultas : BINUS Online Learning
Jenis Karya : Skripsi

Demi pengembangan ilmu pengetahuan, menyetujui untuk memberikan kepada Universitas Bina Nusantara **Hak Prediktor Royalti Noneksklusif** (*Non-exclusive Royalty-Free Right*) atas tugas akhir saya yang berjudul:

”OPTIMASI SISTEM HMI SUMBER TERBUKA PADA PLC OMRON BERBASIS PROTOKOL FINS PADA PT XYZ”

beserta perangkat yang ada (jika diperlukan). Terkait dengan hal diatas, maka pihak Universitas Bina Nusantara berhak menyimpan, mengalih-media/format-kan, mengola dalam bentuk pangkalan data (*database*), merawat dan mempublikasikan tugas akhir saya selama tetap mencantumkan nama saya sebagai penulis/pencipta dan sebagai pemilik Hak Cipta.

Demikian surat pernyataan ini saya buat dengan sebenarnya.

Dibuat di Jakarta pada 2025

Yang menyatakan

()

ABSTRAK

Berbagai macam akselerator telah dikembangkan dalam meningkatkan kinerja dan efisiensi energi untuk menangani komputasi berat, salah satu diantaranya yaitu FPGA. FPGA mampu menangani beban komputasi yang begitu berat sehingga dapat digunakan untuk *Digital Signal Processing*, *Image Processing*, *Neural Network*, dan sebagainya. Pada penelitian ini penulis mencoba mengkaji kinerja yang dimiliki ARM prosesor dan FPGA pada *FPGA Development Board* Xilinx PYNQ Z2 dalam penerapan filter spasial linear pada *video stream*. Kernel filter yang digunakan pada penelitian ini yaitu *average blur*, *gaussian blur*, *laplacian*, *sharpen*, *sobel horizontal* dan *sobel vertical*. Parameter yang digunakan untuk mengukur kinerja ARM prosesor dan FPGA yaitu waktu komputasi, *frame rate* (FPS), penggunaan CPU, penggunaan *memory*, *resident memory* (RES), *shared memory* (SHR) dan *virtual memory* (VIRT). Rata-rata waktu komputasi yang dibutuhkan untuk menerapkan filter spasial linear pada 200 frame dengan ARM prosesor adalah 29.06 detik sedangkan dengan FPGA rata-rata hanya dibutuhkan 3.32 detik. Waktu komputasi dengan FPGA 88.85% lebih baik dibandingkan dengan ARM prosesor. Video hasil filter dengan ARM prosesor memperoleh rata-rata 6.95 fps sedangkan dengan FPGA rata-rata 60.37 fps. FPS dengan FPGA 88.49% lebih baik lebih baik dibandingkan ARM prosesor. Penggunaan CPU pada FPGA 14.89% lebih baik, penggunaan *memory* pada FPGA 2.02% lebih baik, penggunaan *resident memory* 2.07% lebih baik, dan penggunaan *shared memory* 4.08% lebih baik dibandingkan dengan ARM prosesor. Sedangkan penggunaan *virtual memory* pada ARM prosesor 0.03% lebih baik dibandingkan FPGA.

Kata Kunci : filter spasial linear, FPGA, ARM prosesor, *video stream*, *video processing*

ABSTRACT

Various kinds of accelerators have been developed to improve performance and energy efficiency to handle heavy computations, one of which is FPGA. FPGA is capable of handling such a heavy computational load that it can be used for Digital Signal Processing, Image Processing, Neural Networks, etc. In this study, the authors tried to examine the performance of the ARM processor and the FPGA on the Xilinx PYNQ Z2 FPGA Development Board in applying a linear spatial filter to the video stream. Kernel filters used in this study are the average blur, Gaussian blur, Laplacian, sharpen, Sobel horizontal, and Sobel vertical. The parameters used to measure the performance of ARM processors and FPGAs are runtime, frame rate (FPS), CPU usage, memory usage, resident memory (RES), shared memory (SHR), and virtual memory (VIRT). The average computation time required to apply linear spatial filters to 200 frames with an ARM processor is 29.06 seconds, while the average FPGA takes only 3.32 seconds. Compute time with FPGA is 88.85% better than ARM processor. The filtered video with the ARM processor gets an average of 6.95 fps while the FPGA average is 60.37 fps. FPS with FPGA is 88.49% better than ARM processor. CPU usage on FPGA is 14.89% better, memory usage on FPGA is 2.02% better, usage of resident memory is 2.07% better, and usage of shared memory is 4.08% better than ARM processor. While the use of virtual memory on ARM processors is 0.03% better than FPGA.

Keywords : linear spatial filter, FPGA, ARM processor, video stream, video processing

DAFTAR ISI

DAFTAR TABEL

DAFTAR GAMBAR

DAFTAR LAMPIRAN

BAB I

PENDAHULUAN

1.1 Latar Belakang

Programmable logic controller (PLC) merupakan perangkat utama dalam otomasi industri. Fungsi utama dari PLC adalah memproses dan mengendalikan mesin dengan pengetahuan yang terbatas mengenai pemrograman komputer (**koondhar2023role**). Di lingkungan PT XYZ, perusahaan manufaktur komponen otomotif multinasional, sebagian besar mesin produksi telah menggunakan PLC merek Omron. Data internal menunjukkan bahwa lebih dari 90% mesin produksi pada perusahaan mengandalkan PLC Omron sebagai pusat kendali. Tingginya tingkat adopsi PLC Omron pada lini produksi menjadikannya objek penelitian yang relevan dalam konteks integrasi sistem kendali dengan teknologi digital modern.

Untuk mendukung interaksi antara operator dan mesin yang dikendalikan oleh PLC, dibutuhkan *human-machine interface* (HMI). HMI berperan menampilkan data proses, memberikan kontrol interaktif, serta mendukung pengawasan visual yang memudahkan manajemen produksi (**mujawar2023'hmi'intro**). Selama ini, banyak industri mengandalkan sistem HMI komersial. Sistem HMI komersial memiliki keunggulan dalam pengendalian, pengawasan, dan analisis untuk mengurangi *downtime*, biaya, serta menjaga standar kualitas (Sawal, 2025). Namun, ketergantungan pada vendor tertentu sering menimbulkan masalah biaya lisensi, keterbatasan fleksibilitas, dan risiko *vendor lock-in* (Gularte, 2025). Sebagai alternatif, pendekatan sumber terbuka (*open source*) menawarkan efisiensi biaya, fleksibilitas, serta kemudahan kustomisasi. Salah satu platform HMI/SCADA *open source* yang berkembang adalah FUXA, yang berbasis web, ringan, serta mendukung akses jarak jauh dan integrasi IoT (Chen, 2025). Karakteristik ini menjadikan FUXA sebagai sistem HMI sumber terbuka yang potensial untuk mendukung kebutuhan otomasi cerdas di industri manufaktur.

Pada PLC Omron, komunikasi data umumnya dilakukan menggunakan protokol *factory interface network service* (FINS), yakni standar komunikasi yang memungkinkan pertukaran data antarperangkat secara cepat dan andal. Protokol

ini memiliki peran penting dalam menghubungkan PLC dengan sistem HMI, sehingga data proses dapat ditampilkan secara visual dan operator dapat melakukan pengendalian secara efektif. Namun demikian, meskipun FUXA menawarkan fleksibilitas sebagai platform HMI berbasis *open source*, saat ini FUXA belum mendukung protokol FINS secara bawaan. Keterbatasan ini menjadi kendala utama dalam implementasi integrasi FUXA dengan PLC Omron yang digunakan pada lini produksi.

Untuk mengatasi kendala tersebut, diperlukan upaya pengembangan dengan menambahkan dukungan protokol FINS pada FUXA melalui modifikasi kode sumbernya. Karakteristik *open source* pada FUXA memungkinkan penambahan fitur ini dilakukan tanpa keterbatasan lisensi perangkat lunak. Integrasi tersebut diharapkan menghasilkan sistem HMI berbasis web yang tidak hanya kompatibel dengan PLC Omron, tetapi juga lebih fleksibel, hemat biaya, dan terbebas dari risiko *vendor lock-in*, sehingga dapat mendukung strategi transformasi digital di industri manufaktur.

Berdasarkan uraian di atas, penelitian ini difokuskan pada optimasi sistem HMI sumber terbuka berbasis web yang terintegrasi dengan PLC Omron melalui protokol FINS. Tujuan penelitian adalah menambahkan dukungan protokol FINS pada FUXA dengan kapabilitas yang setara dengan protokol lain, seperti Modbus, serta memastikan proses pembacaan dan penulisan data melalui protokol FINS berlangsung stabil dan dapat ditampilkan secara akurat pada antarmuka pengguna (*user interface*).

1.2 Identifikasi Masalah

Berdasarkan uraian latar belakang di atas, dapat diidentifikasi beberapa permasalahan yang dihadapi PT XYZ terkait kebutuhan sistem SCADA, yaitu:

1. Sistem HMI belum mendukung protokol komunikasi FINS secara bawaan, sehingga tidak dapat langsung digunakan untuk integrasi dengan PLC Omron.
2. Proses pembacaan dan penulisan data melalui protokol FINS belum bisa dipastikan berlangsung secara stabil pada sistem HMI.

1.3 Rumusan Masalah

Rumusan masalah dalam penelitian ini adalah sebagai berikut:

1. Bagaimana menambahkan fitur protokol FINS pada sistem HMI dengan fitur yang setara dengan protokol lain seperti Modbus.
2. Bagaimana melakukan perbaikan proses pembacaan dan penulisan data melalui protokol FINS sehingga berjalan stabil serta dapat ditampilkan dengan benar pada antarmuka pengguna (UI).

1.4 Tujuan Penelitian

Tujuan dari penelitian ini adalah untuk:

1. Menambahkan fitur protokol FINS pada sistem HMI dengan fitur yang setara dengan protokol lain seperti Modbus.
2. Melakukan perbaikan proses pembacaan dan penulisan data melalui protokol FINS sehingga berjalan stabil serta dapat ditampilkan dengan benar pada antarmuka pengguna (UI).

1.5 Manfaat Penelitian

Manfaat dari penelitian ini antara lain:

- Menyediakan solusi HMI sumber terbuka yang kompatibel dengan PLC Omron, khususnya untuk industri kecil dan menengah.
- Memberikan kontribusi nyata dalam pengembangan perangkat lunak sumber terbuka di bidang otomasi industri.

1.6 Ruang Lingkup Penelitian

Penelitian ini memiliki ruang lingkup sebagai berikut:

- Fokus pada implementasi komunikasi FINS melalui protokol UDP/TCP.

- Penggunaan area memori standar PLC Omron seperti DM, CIO, W, dan sejenisnya.
- Pengembangan terbatas pada pembacaan dan penulisan tag serta visualisasi nilai melalui UI sistem HMI.
- Pengujian dilakukan menggunakan perangkat lunak analisis jaringan seperti Wireshark serta PLC fisik maupun simulator.

1.7 Hipotesis Penelitian

Hipotesis dari penelitian ini adalah:

- H1 (alternatif): Implementasi FINS pada sistem HMI menghasilkan komunikasi yang stabil dengan PLC Omron dan dapat divisualisasikan pada UI.
- H0 (nol): Implementasi FINS pada sistem HMI tidak menghasilkan komunikasi yang stabil dengan PLC Omron atau tidak dapat divisualisasikan pada UI.

1.8 Sistematika Penulisan

Adapun sistematika penulisan dalam laporan ini adalah sebagai berikut:

- **Bab 1** – Pendahuluan: berisi latar belakang, rumusan masalah, tujuan dan manfaat, ruang lingkup, hipotesis, dan sistematika penulisan.
- **Bab 2** – Tinjauan Referensi: membahas teori dan referensi terkait, seperti protokol FINS, sistem HMI, dan komunikasi industri.
- **Bab 3** – Metodologi Penelitian: menjelaskan tahapan dan metode penelitian yang digunakan.
- **Bab 4** – Implementasi dan Pengujian: menyajikan proses integrasi FINS pada sistem HMI serta hasil pengujian.
- **Bab 5** – Kesimpulan dan Saran: berisi simpulan dari hasil penelitian serta rekomendasi untuk pengembangan lebih lanjut.

BAB II

TINJAUAN PUSTAKA

2.1 Tinjauan Studi

Tinjauan studi berisi rangkuman penelitian-penelitian terdahulu yang dijadikan acuan dalam menyusun penelitian ini. Tinjauan tersebut diharapkan dapat memperluas wawasan serta memperdalam pemahaman terhadap teori-teori yang telah dikembangkan sebelumnya. Adapun daftar penelitian terdahulu yang menjadi rujukan dalam penelitian ini disajikan pada Tabel 2.1.

Tabel 2.1: Tabel Tinjauan Studi

No.	Penulis (Tahun)	Judul Artikel	Domain Aplikasi	Platform / Teknologi	Tujuan / Kontribusi
1	Uddin dkk. (2022)	Design and Implementation of an Open-Source SCADA System for a Community Solar-Powered Reverse Osmosis System	Industri Air Bersih	Node-RED, Grafana, InfluxDB, Debian OS, Protokol Firmata, Arduino Mega 2560, Arsitektur SCADA Sumber Terbuka	Merancang dan mengimplementasikan SCADA sumber terbuka untuk sistem desalinasi berbasis tenaga surya. Kontribusi: solusi murah dan terbuka untuk komunitas kecil seperti pedesaan.

No.	Penulis (Tahun)	Judul Artikel	Domain Aplikasi	Platform / Teknologi	Tujuan / Kontribusi
2	Omidi dkk. (2023)	Design and Implementation of Node-Red Based Open-Source SCADA Architecture for a Hybrid Power System	Sistem Pembangkit Listrik Terbarukan Hibrida	Node-RED, Grafana, InfluxDB, MQTT, Firmata, ESP32, Raspberry Pi	Mengembangkan SCADA sumber terbuka untuk sistem pembangkit listrik hibrida. Kontribusi: sistem modular, berdaya rendah, dan biaya rendah.

No.	Penulis (Tahun)	Judul Artikel	Domain Aplikasi	Platform / Teknologi	Tujuan / Kontribusi
3	Almas dkk. (2014)	Open Source SCADA Implementation and PMU Integration for Power System Monitoring and Control Applications	Sistem Tenaga Listrik	SCADA BR, PMU, DNP3, RT-HIL, LabVIEW, Statnett SDK, Pachube	Mengembangkan SCADA BR dan integrasi PMU. Kontribusi: evaluasi sistem tenaga waktu nyata dan batasan polling DNP3.
4	Salazar dkk. (2024)	ICSNet: A Hybrid-Interaction Honeynet for Industrial Control Systems	Keamanan Siber Sistem Kendali Industri	FUXA HMI, Honeyd, ICSNet, Factory I/O, Mininet, Modbus TCP, IEC-104, ENIP, SNMP, Nmap, Nikto, Proxy	Mengembangkan ICSNet sebagai honeynet hibrida realistis dan modular untuk ICS. Kontribusi: integrasi honeypot interaksi rendah dan tinggi.

No.	Penulis (Tahun)	Judul Artikel	Domain Aplikasi	Platform / Teknologi	Tujuan / Kontribusi
5	Hazdi dkk. (2017)	Desain SCADA Berbasis Web untuk Otomasi Rumah	Otomatisasi Rumah	PLC, SCADA berbasis web, HMI, OPC, MySQL, Visual Basic, Sensor Arus, Jaringan Ethernet	Mengembangkan SCADA untuk otomatisasi rumah dan mengujinya dalam parameter waktu. Kontribusi: aplikasi SCADA dalam rumah tangga dan penyediaan data eksperimental.
6	Kasun Thushara (2024)	Getting Start with FUXA - Web Based SCADA Tool	SCADA/HMI Industri	FUXA HMI, reTerminal DM, Modbus, MQTT, Debian OS	Mendiskusikan implementasi FUXA SCADA berbasis web di perangkat edge. Kontribusi: solusi murah dan terbuka untuk sistem HMI/SCADA industri.

2.2 Tinjauan Pustaka

2.2.1 Objek Penelitian

Objek penelitian adalah PLC Omron seri CJ2M yang berkomunikasi melalui protokol Factory Interface Network Service (FINS). PLC ini digunakan secara luas di industri manufaktur di Indonesia, termasuk di PT XYZ, sehingga mendukung pentingnya penelitian ini.

Selain itu, perangkat lunak open-source FUXA dipilih sebagai platform HMI berbasis web karena berbasis Node.js dan Angular, mendukung Modbus, Siemens S7, BACnet, OPC UA, dan dapat diperluas dengan protokol baru seperti FINS.

2.2.2 Perusahaan Manufaktur dan Kebutuhan Otomasi

Industri manufaktur modern sangat bergantung pada sistem otomasi untuk meningkatkan efisiensi produksi, konsistensi kualitas, dan pengendalian biaya. Otomasi industri melibatkan penggunaan perangkat keras seperti sensor, aktuator, dan Programmable Logic Controller (PLC) yang terhubung ke sistem kontrol dan pengawasan seperti HMI/SCADA (Supervisory Control and Data Acquisition) (McKinsey & Company, 2023).

Perusahaan seperti *Special Purpose Machine (SPM) Maker* sering kali merancang dan membangun mesin otomatis untuk kebutuhan spesifik di pabrik. Mesin-mesin ini biasanya menggunakan PLC dari berbagai vendor, termasuk Omron, Siemens, dan Allen-Bradley. Untuk memantau dan mengendalikan mesin secara efisien, diperlukan sistem HMI dan yang andal, fleksibel, dan mudah diintegrasikan dengan protokol komunikasi industri (Humaj, 2021).

2.2.3 Programmable Logic Controller (PLC)

PLC adalah perangkat digital berbasis mikroprosesor yang dirancang untuk mengontrol proses otomatis di lingkungan industri. PLC dapat diprogram untuk menjalankan logika kontrol yang kompleks dan sangat tahan terhadap kondisi ekstrem seperti getaran, suhu tinggi, dan interferensi listrik. PLC berfungsi sebagai otak dari sistem otomasi, mengumpulkan data dari sensor dan mengontrol aktuator berdasarkan

logika yang telah diprogram (EMQX, 2023).

2.2.4 Protokol Komunikasi Industri dan FINS

Agar PLC dapat terhubung ke perangkat lain, diperlukan protokol komunikasi industri. Protokol ini memungkinkan transfer data secara waktu nyata antara PLC dan HMI. Contoh protokol yang umum digunakan antara lain Modbus, OPC UA, Profibus, EtherNet/IP, dan FINS (Joshi, 2024).

FINS (Factory Interface Network Service) merupakan protokol yang dikembangkan oleh Omron untuk memungkinkan komunikasi antar perangkat di dalam jaringan otomasi industri (Humaj, 2021). FINS mendukung komunikasi melalui UDP dan TCP. Kelebihan FINS antara lain:

- Kompatibel dengan semua seri PLC Omron.
- Dukungan komunikasi jarak jauh melalui pengalamatan jaringan.
- Struktur data fleksibel, seperti area CIO, DM, WR, HR.

2.2.5 HMI dan Visualisasi Data

HMI (Human-Machine Interface) adalah antarmuka antara manusia dan sistem kontrol. HMI menyediakan representasi visual dari proses industri dan memungkinkan operator untuk memantau kondisi sistem dan melakukan intervensi jika diperlukan. Fitur penting dalam HMI meliputi grafik waktu nyata, pengaturan parameter, tampilan alarm, dan trend historis (Thushara, 2024).

2.2.6 Teknologi Web Modern: HTML, CSS, JavaScript, TypeScript, Node.js, Angular

Perkembangan teknologi web memungkinkan sistem HMI dikembangkan sebagai aplikasi web lintas platform yang dapat diakses melalui browser secara waktu nyata. Teknologi utama yang digunakan antara lain:

- **HTML (HyperText Markup Language):** Bahasa standar untuk menyusun struktur dan elemen-elemen dasar halaman web. Bhanarkar et al. (2023).

- **CSS (Cascading Style Sheets):** Digunakan untuk mendesain tampilan visual halaman web, termasuk warna, tata letak, dan responsivitas antarmuka. Singh (2014).
- **JavaScript:** Bahasa pemrograman inti untuk interaktivitas pada web, memungkinkan manipulasi DOM, penanganan event, dan komunikasi asynchronous melalui AJAX. Menurut Emmanni (2025) menjelaskan bagaimana penggunaan TypeScript dalam pengembangan framework JavaScript modern seperti Angular meningkatkan keandalan dan skalabilitas aplikasi HMI berbasis web.
- **TypeScript:** Merupakan superset dari JavaScript yang dikembangkan oleh Microsoft, menyediakan fitur pengetikan statis dan pemrograman berorientasi objek yang kuat. TypeScript digunakan secara luas dalam pengembangan Angular karena meningkatkan skalabilitas, keamanan, dan maintainability kode. Menurut Scarsbrook et al. (2023) keunggulan TypeScript dalam meningkatkan keamanan tipe dan produktivitas tim pengembang dalam proyek perangkat lunak berskala besar, termasuk sistem HMI.
- **Node.js:** Platform berbasis JavaScript yang berjalan di sisi server (backend). Node.js mendukung arsitektur non-blocking dan event-driven, sehingga sangat cocok untuk aplikasi HMI yang membutuhkan performa tinggi dan komunikasi data waktu nyata. Menurut Ancona et al. (2018) menjelaskan bahwa Node.js memiliki karakteristik yang mendukung pemantauan sistem secara waktu nyata, menjadikannya kandidat yang sesuai untuk penerapan pada sistem Internet of Things (IoT) dan HMI modern.
- **Angular:** Framework frontend modern yang dikembangkan oleh Google. Angular menggunakan TypeScript sebagai bahasa utamanya dan menyediakan pendekatan pengembangan berbasis komponen, dependency injection, serta routing yang efisien. Angular mempermudah pembuatan antarmuka pengguna (HMI) yang dinamis, modular, dan responsif. Menurut Client IO (2023), yang memanfaatkan teknologi frontend modern termasuk Angular untuk membangun antarmuka HMI interaktif.

Dengan kombinasi teknologi tersebut, sistem HMI berbasis web menjadi lebih fleksibel, ringan, dan dapat diakses lintas perangkat tanpa memerlukan instalasi perangkat lunak tambahan (Thushara, 2024).

2.2.7 HMI Open-Source dan Vendor Lock-In

Salah satu tantangan utama dalam dunia industri adalah ketergantungan terhadap vendor atau *vendor lock-in*. Sistem HMI komersial umumnya bersifat tertutup dan berlisensi mahal, sehingga membatasi fleksibilitas pengguna dalam hal integrasi, migrasi, maupun penyesuaian sistem.

Vendor lock-in menyebabkan pengguna hanya dapat menggunakan layanan, protokol, atau perangkat lunak dari penyedia tertentu, sehingga sulit untuk beralih ke penyedia lain tanpa mengeluarkan biaya besar atau menghadapi gangguan layanan. Dalam konteks cloud computing, fenomena ini bahkan menjadi lebih kritis, karena banyak organisasi bergantung pada layanan seperti DBaaS (Database-as-a-Service). Ketika organisasi membutuhkan fitur baru atau terjadi perubahan harga, ketergantungan ini menjadi hambatan besar untuk berpindah platform (Banthia, 2024).

Untuk mengatasi vendor lock-in, banyak organisasi mulai mengadopsi pendekatan berbasis teknologi terbuka dan strategi multi-cloud. Salah satu solusi yang relevan dalam konteks HMI adalah penggunaan sistem open-source seperti FUXA. Dengan menggunakan HMI open-source, pengguna memperoleh kendali penuh atas kode sumber, arsitektur, dan kemampuan integrasi sistem, sehingga meminimalkan risiko dikunci oleh vendor tertentu.

FUXA adalah contoh HMI berbasis web yang bersifat open-source dan mendukung berbagai protokol industri seperti Modbus, OPC-UA, dan MQTT. Sistem ini dapat dikustomisasi secara penuh dan tidak tergantung pada penyedia perangkat lunak tertentu (Thushara, 2024).

2.2.8 Electron: Aplikasi Desktop Berbasis Web

Electron adalah framework open-source yang memungkinkan pengembangan aplikasi desktop lintas platform (Windows, macOS, dan Linux) menggunakan

teknologi web seperti HTML, CSS, dan JavaScript, dikombinasikan dengan Node.js dan Chromium. Framework ini banyak digunakan untuk membangun aplikasi desktop modern karena kemampuannya menjalankan kode frontend dan backend dalam satu kerangka kerja yang terintegrasi (Electron Team, 2024).

Beberapa keunggulan utama Electron antara lain:

- **Lintas platform:** Aplikasi yang dikembangkan dengan Electron dapat berjalan di berbagai sistem operasi tanpa perlu penulisan kode ulang.
- **Teknologi web modern:** Memanfaatkan ekosistem luas dari JavaScript, TypeScript, dan pustaka web.
- **Stabil dan enterprise-grade:** Digunakan oleh perusahaan besar seperti Microsoft (Visual Studio Code), OpenAI (ChatGPT Desktop), Discord, dan Slack.
- **Kemudahan integrasi native:** Mendukung pemanggilan kode native (C++, Rust, dsb) jika diperlukan.

Dengan bundling Chromium dan Node.js, Electron memberikan kendali penuh terhadap tampilan dan fungsionalitas aplikasi desktop, sekaligus menghindari keterbatasan atau bug dari webview bawaan sistem operasi (Electron Team, 2024).

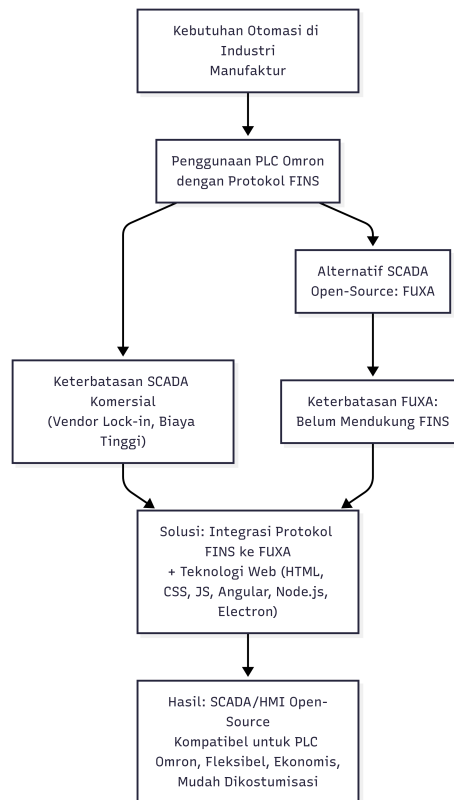
2.2.9 Pengujian dan Validasi Sistem HMI

Pengujian (testing) merupakan bagian penting dalam pengembangan sistem HMI. Pengujian bertujuan untuk memastikan sistem dapat membaca dan menulis data secara benar, menangani kondisi ekstrem, serta menampilkan visualisasi data yang akurat. Pengujian biasanya mencakup:

- **Unit testing:** Memastikan setiap komponen bekerja sesuai fungsi.
- **Integration testing:** Memastikan komunikasi antara komponen berjalan lancar.
- **System testing:** Menguji sistem secara menyeluruh dalam kondisi nyata atau simulasi.

- **Network traffic analysis:** Menggunakan Wireshark atau alat sejenis untuk memantau paket FINS dalam jaringan (Almas & Vanfretti, 2014).

2.3 Kerangka Pemikiran



Gambar 2.1: Diagram kerangka pemikiran penelitian

Berdasarkan kajian pustaka yang telah dibahas pada bagian 2.2, penelitian ini berangkat dari kebutuhan otomatisasi di industri manufaktur yang semakin tinggi seiring dengan perkembangan teknologi digital dan penerapan konsep Industri 4.0. PT XYZ sebagai perusahaan manufaktur komponen otomotif menggunakan *Programmable Logic Controller (PLC)* Omron yang berkomunikasi melalui protokol *Factory Interface Network Service (FINS)* untuk mengendalikan sebagian besar mesin produksinya. Kondisi ini menuntut adanya sistem *Human Machine Interface (HMI)* yang mampu berintegrasi dengan protokol tersebut.

Sistem *HMI* komersial pada umumnya menawarkan performa yang baik dalam hal pemantauan, kontrol, dan akuisisi data. Namun, sistem ini seringkali menimbulkan

permasalahan *vendor lock-in* yang menyebabkan perusahaan harus bergantung pada penyedia tertentu dengan konsekuensi keterbatasan fleksibilitas integrasi. Untuk mengatasi hal tersebut, solusi berbasis sumber terbuka seperti *FUXA* dipandang lebih sesuai karena memberikan fleksibilitas, efisiensi biaya, serta kemampuan kustomisasi sesuai kebutuhan perusahaan.

Meskipun demikian, *FUXA* belum mendukung protokol *FINS* secara bawaan, sehingga tidak dapat langsung digunakan untuk berkomunikasi dengan *PLC Omron* di PT XYZ. Oleh karena itu, penelitian ini berfokus pada pengembangan integrasi protokol *FINS* ke dalam *FUXA*, dengan memanfaatkan teknologi *web* modern seperti *HTML*, *CSS*, *JavaScript*, *TypeScript*, *Angular*, dan *Node.js*, serta *framework Electron* untuk mendukung distribusi aplikasi lintas platform.

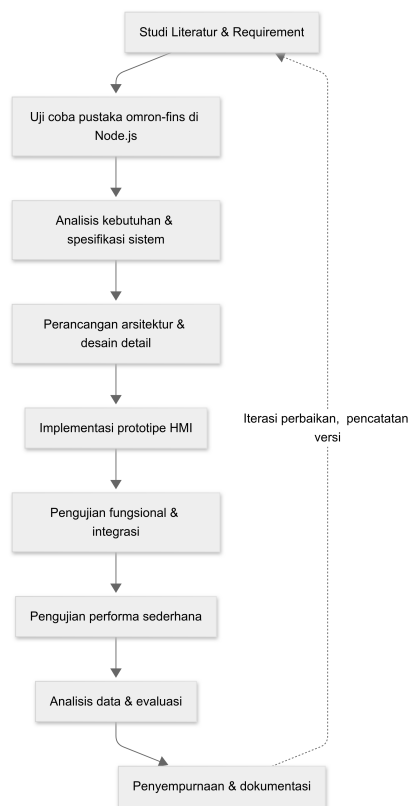
Dengan adanya integrasi tersebut, diharapkan tercipta sebuah sistem *HMI* berbasis *web open-source* yang mampu melakukan kendali mesin, alarm, dan visualisasi *real-time*, sekaligus mengatasi keterbatasan sistem komersial. Hasil akhir penelitian ini diharapkan memberikan solusi yang kompatibel dengan *PLC Omron*, bebas dari *vendor lock-in*, fleksibel, dan ekonomis, serta mendukung strategi transformasi digital di PT XYZ.

BAB III

METODOLOGI PENELITIAN

3.1 Tahapan Penelitian

Tahapan penelitian dijalankan secara iteratif dengan pendekatan *iterative prototyping*, yaitu siklus berulang antara analisis, desain, implementasi, pengujian, dan evaluasi. Diagram alir (flowchart) tahapan penelitian disajikan pada Gambar ??.



Gambar 3.1: Flowchart Tahapan Penelitian

Tahapan rinci penelitian adalah sebagai berikut:

- 1. Studi literatur dan perumusan requirement**

Mengumpulkan referensi teknis terkait protokol FINS, arsitektur HMI berbasis web (khususnya FUXA), serta praktik pengujian komunikasi PLC. Hasil: dokumen requirement fungsional dan non-fungsional.

- 2. Uji coba pustaka omron-fins di Node.js**

Membuat skrip sederhana (`Communication test .js`) untuk menguji apakah pustaka dapat terkoneksi dengan PLC Omron, melakukan operasi baca, dan menulis nilai ke alamat tertentu. Hasil: validasi awal apakah pustaka berfungsi dengan PLC target.

3. Analisis kebutuhan dan spesifikasi sistem

Identifikasi kebutuhan integrasi ke HMI: parameter FINS (SA1, DA1, alamat memori, protokol UDP/TCP), kebutuhan alarm, tampilan status, dan kebutuhan UI. Hasil: spesifikasi teknis dan use-case.

4. Perancangan arsitektur dan desain detail

Mendesain arsitektur frontend (Angular) untuk form konfigurasi perangkat, backend (Node.js) untuk modul FINS, serta jalur komunikasi antara keduanya. Hasil: diagram arsitektur dan desain skema komunikasi.

5. Implementasi prototipe HMI

Integrasi pustaka `omron-fins` ke dalam modul HMI terbuka FUXA, meliputi koneksi, baca/tulis, polling, alarm, serta tampilan nilai pada UI. Hasil: prototipe fungsional HMI dengan dukungan FINS.

6. Pengujian fungsional dan integrasi

Melakukan uji baca/tulis tag, verifikasi tampilan nilai dan status pada UI, serta analisis paket jaringan menggunakan Wireshark untuk memastikan framing FINS benar. Hasil: laporan bug dan perbaikan iteratif.

7. Pengujian performa sederhana

Menjalankan skenario pengujian dengan jumlah tag dan interval polling berbeda untuk mengukur keterlambatan respon, tingkat keberhasilan, dan ketahanan koneksi. Hasil: dataset pengujian (CSV, log) dan metrik evaluasi.

8. Analisis data dan evaluasi

Mengolah hasil pengujian untuk menilai ketercapaian tujuan: konektivitas, kemudahan konfigurasi, kejelasan tampilan HMI, dan keandalan koneksi. Hasil: analisis dan rekomendasi optimasi.

9. Penyempurnaan dan dokumentasi

Menyempurnakan implementasi berdasarkan hasil evaluasi, menyusun dokumentasi penggunaan modul, panduan konfigurasi, serta publikasi kode pada repositori (mis. GitHub). Hasil: rilis final modul dan panduan.

Pada setiap iterasi siklus di atas, dilakukan evaluasi risiko teknis (mis. *timeout*, *connection error*), mitigasi, dan pencatatan perubahan versi. Tahapan ini memastikan bahwa integrasi FINS pada HMI yang dikembangkan tidak hanya berfungsi secara fungsional tetapi juga andal (reliable) dan terdokumentasi.

3.2 Alat dan Bahan

3.2.1 Perangkat Keras

- Laptop (Intel i7, RAM 16GB, SSD 512GB, OS Windows11).
- Industrial Switching Hub Omron W4S1-05B.
- PLC Omron CJ2M CPU34.

3.2.2 Perangkat Lunak

- Node.js v20.x, Angular v17.x, Electron v28.x.
- Wireshark v4.x untuk analisis paket.
- Visual Studio Code, Git.

3.2.3 Library Tambahan

- `node-omron-fins`.
- Angular Material untuk komponen UI konfigurasi.

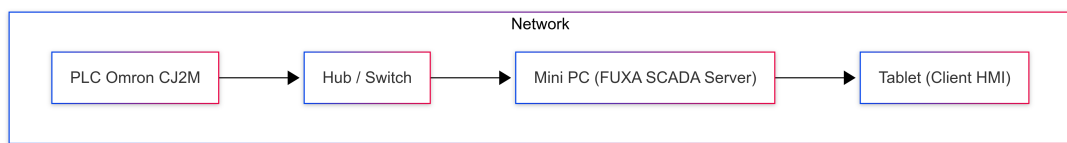
3.2.4 Platform dan Arsitektur Aplikasi

Aplikasi dikembangkan menggunakan framework **Electron**, yang mengemas frontend Angular dan backend Node.js ke dalam executable lintas platform.

Keunggulan pendekatan ini antara lain:

- Distribusi mudah ke Windows dan Linux.
- Performa tinggi dengan mesin V8 (Chromium).
- Akses langsung ke sistem file/logging lokal.
- Portabilitas tinggi di lingkungan industri.

3.2.5 Topologi Sistem dan Komunikasi FINS



Gambar 3.2: Topologi Sistem SCADA FUXA dengan Protokol FINS (Mini PC, PLC, Hub, Tablet)

Pada arsitektur ini, **PLC Omron CJ2M** terhubung ke sebuah **hub/switch** yang berfungsi sebagai pengatur lalu lintas jaringan. **Mini PC** bertindak sebagai pusat kendali sekaligus server HMI , yang menjalankan konektor FINS untuk komunikasi dengan PLC. Mini PC kemudian dapat diakses oleh **tablet** melalui jaringan yang sama, sehingga operator dapat melakukan monitoring dan kontrol secara real-time melalui antarmuka web FUXA. Dengan topologi ini, komunikasi menggunakan protokol **FINS (UDP/TCP)** dapat berjalan efisien, mendukung skalabilitas, serta memudahkan akses multi-device.

3.3 Modifikasi UI (Frontend Angular)

The screenshot shows a 'Connections Property' dialog box with a close button (X) in the top right corner. The dialog contains the following fields and controls:

- Name:** A text input field containing 'PLC_Mesin_Cutting'.
- Type:** A dropdown menu with 'Fins' selected.
- Polling:** A dropdown menu with '350 ms' selected.
- Enable:** A toggle switch that is currently turned on.
- FINS IP Address:** A text input field containing '192.168.0.1'.
- SA1 (PC Node):** A text input field containing '234'.
- DA1 (PLC Node):** A text input field containing '1'.
- Protocol:** A dropdown menu with 'UDP' selected.

At the bottom right of the dialog are two buttons: 'CANCEL' and 'OK'.

Gambar 3.3: Rencana modifikasi antarmuka pengguna (UI) untuk konfigurasi perangkat FINS

Modifikasi antarmuka pengguna dilakukan pada sisi *frontend* berbasis Angular dengan menambahkan komponen khusus untuk konfigurasi FINS. Perubahan utama antara lain:

- Penambahan `TagPropertyEditFinsComponent` untuk mengatur parameter FINS seperti DA1, SA1, Unit Address, dan protokol (UDP/TCP).
- Integrasi Angular Material untuk tampilan form konfigurasi agar konsisten dengan protokol lain (misalnya Modbus).
- Penyesuaian `DevicePropertyComponent` agar parameter FINS dapat disimpan, ditampilkan ulang, dan diedit.
- Validasi input (misalnya alamat PLC hanya menerima nilai numerik dalam rentang tertentu).

Tag Property [X]

Device
dLC_mesin_bubutayam

Tagname
sda

Tegister
tombol_mesin_start ▼

Type
Int16 ▼

Address offset (1-65536)
4500

Divisor

Description

CANCEL OK

Gambar 3.4: Rencana modifikasi antarmuka pengguna (UI) untuk konfigurasi tag perangkat FINS

Selain perencanaan awal (Gambar ??), implementasi nyata form edit tag ditunjukkan pada Gambar ??. Antarmuka ini memungkinkan pengguna untuk menambahkan atau mengubah konfigurasi tag FINS secara langsung melalui dialog khusus, termasuk:

- Pemilihan area memori (CIO, DM, WR, HR).
- Input alamat numerik sesuai range yang didukung PLC Omron.
- Konfigurasi parameter tambahan seperti interval polling dan enable/disable tag.
- Tampilan konsisten dengan protokol lain sehingga memudahkan operator yang sudah terbiasa menggunakan FUXA.

3.4 Modifikasi Backend (Node.js Konektor FINS)

Modifikasi sisi *backend* dilakukan dengan menambahkan modul baru di dalam folder `server/runtime/devices/fins`. Perubahan meliputi:

- Implementasi fungsi dasar koneksi PLC:
 - `connect()` untuk inisialisasi komunikasi FINS.
 - `read()` untuk membaca nilai alamat PLC.
 - `write()` untuk menulis nilai ke alamat PLC.
 - `poll()` untuk melakukan polling berkala.
- Mekanisme `retry` jika koneksi terputus atau terjadi *timeout*.
- Pengaturan logging error agar memudahkan debugging.
- Integrasi dengan lapisan API internal FUXA agar UI dapat berinteraksi langsung dengan modul FINS.

Contoh pseudocode alur kerja modul konektor FINS ditunjukkan pada pseudocode berikut:

Pseudocode: DeviceFins Module

1. LOAD library "omron-fins"

IF gagal -> log error, modul tidak tersedia

2. FUNCTION DeviceFins(data, logger, events, runtime):

INIT:

client = null

values = {}

isConnected = false

isConnecting = false

reconnectTimer = null

```

deviceId = data.id
deviceName = data.name
deviceTags = list dari data.tags
options = data.property

host = options.address OR default
port = options.port OR default
protocol = options.FinsProtocol OR 'UDP'
SA1, DA1 = options.SA1, options.DA1

```

LOG konfigurasi device

FUNCTION scheduleReconnect():

```

  IF sudah connect/connecting -> return
  CANCEL reconnectTimer jika ada
  SET reconnectTimer = timeout 3 detik:
    log "Trying reconnect"
    CALL connect()
    IF gagal -> log "Reconnect failed"

```

FUNCTION connect():

```

  RETURN Promise
  IF fins library tidak ada -> reject
  isConnected = true
  BERSIHKAN client lama jika ada
  CREATE client baru FinsClient(host, port, SA1, DA1, protocol)
  client.onError:
    log error
    set isConnected=false, isConnected=false
    CALL disconnect()
    CALL scheduleReconnect()

```

```

client.onTimeout:
    log timeout
    set isConnected=false, isConnecting=false
    CALL disconnect()
    CALL scheduleReconnect()
Jika sukses:
    set isConnected=true, isConnecting=false
    emit events "status:connect-ok"
    resolve()

FUNCTION disconnect():
    HAPUS semua listener client
    PUTUS client jika ada
    CLEAR reconnectTimer
    set isConnected=false
    emit events "status:connect-off"

FUNCTION stop() = disconnect

FUNCTION isConnected() RETURN isConnected

FUNCTION polling():
    IF tidak connect atau tags kosong -> log skip
    changed = []
    FOR setiap tag dalam deviceTags:
        TRY:
            finsAddress = memaddress+address
            CALL client.read(finsAddress)
        ON reply:
            val = response.values[0]
            now = timestamp

```



```

        IF val berbeda dari values[tag.id]:
            update values
            tambahkan ke changed
            IF addDaq tersedia:
                simpan data (val, ts)
    ON error:
        log error, set isConnected=false
        CALL scheduleReconnect()
    CATCH err -> log polling exception
IF changed tidak kosong:
    emit events "device-value:changed"

FUNCTION getValues():
    RETURN semua pasangan (id, value)

FUNCTION getValue(tagId):
    RETURN {id, value, ts=now}

FUNCTION getStatus():
    RETURN "connect-ok" jika isConnected else "connect-off"

FUNCTION load(newData):
    jika newData.tags ada -> perbarui deviceTags

FUNCTION setValue(tagId, value):
    cari tag sesuai id
    IF client dan tag ada:
        finsAddress = memaddress+address
        client.write(finsAddress, [value])
        ON error: log gagal tulis
        ELSE: log sukses, update values

```

3. MODULE EXPORT:

`init():` kosong

`create(data,...):` RETURN new DeviceFins

3.5 Metode Pengujian

Pengujian mencakup pengujian fungsional (*black-box*), struktural (*white-box*), serta evaluasi performa.

3.5.1 Black-Box Testing

No	Fitur	Skenario Pengujian	Input	Output yang Diharapkan	Hasil
1	Konfigurasi	Menyimpan parameter FINS (DA1, SA1, Unit Address, protocol).	Nilai parameter konfigurasi	Parameter tersimpan dan dapat dipanggil ulang	Sesuai / Tidak
2	Baca/Tulis Data	Membaca nilai tag PLC dan menulis nilai baru melalui UI.	Alamat memori dan nilai tulis	Nilai tag tampil di UI, nilai tulis terkirim ke PLC	Sesuai / Tidak
3	Alarm	Mengaktifkan kondisi abnormal pada PLC.	Trigger alarm (simulasi fault)	Alarm muncul pada UI	Sesuai / Tidak
4	UI	Mengakses form konfigurasi dan edit tag.	Interaksi user di UI	Tidak ada error dan tampilan sesuai rancangan	Sesuai / Tidak

Tabel 3.1: Black-Box Testing Konektor FINS

3.5.2 White-Box Testing

No	Fitur	Skenario Pengujian	Input	Output yang Diharapkan	Hasil
1	Unit Testing Konektor	Menguji fungsi <code>connect()</code> , <code>read()</code> , <code>write()</code> , <code>poll()</code> .	Skrip uji unit	Fungsi berjalan tanpa error	Sesuai / Tidak
2	Event Handling	Menyimulasikan banyak listener dan memutus koneksi.	Event listener	Listener terputus, tidak ada memory leak	Sesuai / Tidak
3	Error Handling	Putus koneksi jaringan atau ubah IP salah.	Koneksi gagal	Sistem menghubungkan ulang otomatis dan log error muncul	Sesuai / Tidak
4	Traffic Analysis	Capture komunikasi FINS dengan Wireshark.	Paket FINS	Network Paket sesuai spesifikasi FINS	Sesuai / Tidak
5	Integrasi Client-Server	Uji data dari UI Angular ke backend Node.js.	Input nilai dari UI	Data terkirim ke backend dan tersimpan	Sesuai / Tidak

Tabel 3.2: White-Box Testing Konektor FINS

3.5.3 Metrik Evaluasi

Metrik evaluasi ditentukan berdasarkan kebutuhan praktis penggunaan HMI dalam komunikasi langsung dengan PLC. Tabel berikut merangkum metrik dan target kinerja yang diharapkan:

Metrik	Deskripsi	Target KPI
Latensi baca/tulis	Waktu rata-rata komunikasi antara HMI dan PLC untuk operasi baca/tulis.	≤ 100 ms (real-time, tidak terasa delay bagi operator)
Throughput	Jumlah operasi baca/tulis yang dapat diproses per detik.	≥ 100 operasi/detik untuk 100 tag
Persentase keberhasilan komunikasi	Proporsi request yang berhasil dibanding total request.	$\geq 99.5\%$ sukses
Penggunaan CPU/memori	Konsumsi sumber daya oleh aplikasi HMI saat polling aktif.	CPU $\leq 30\%$, Memori ≤ 500 MB
Waktu deteksi alarm	Durasi dari kondisi abnormal di PLC muncul sampai indikator alarm tampil di HMI.	≤ 200 ms
Booting Time	Waktu yang diperlukan aplikasi HMI untuk siap terkoneksi dengan PLC setelah dijalankan.	≤ 10 detik

Tabel 3.3: Metrik dan Target KPI Evaluasi Sistem HMI dengan Konektor FINS

3.5.4 Validasi dan Reproducibility

Untuk memastikan hasil dapat direplikasi:

- Semua kode, konfigurasi, dan skrip pengujian dipublikasikan di GitHub.
- Hasil eksperimen (CSV, log Wireshark) dilampirkan.
- Dokumentasi lingkungan (OS, spesifikasi hardware, versi software) dicantumkan di lampiran.

3.6 Evaluasi Berbasis Responden

Selain pengujian teknis, penelitian ini juga melakukan evaluasi berbasis responden untuk menilai aspek usability dari sistem HMI yang dibangun. Evaluasi dilakukan dengan melibatkan **5–10 responden** yang memiliki latar belakang sebagai mahasiswa otomasi industri, teknisi PLC, atau operator produksi.

Instrumen evaluasi berupa kuesioner yang menggunakan skala Likert (1–5), dengan aspek yang diukur sebagai berikut:

1. **Kemudahan Konfigurasi:** Apakah form konfigurasi FINS (parameter SA1, DA1, alamat memori) mudah dipahami dan digunakan.
2. **Kejelasan Tampilan:** Apakah nilai tag dan status koneksi ditampilkan secara jelas dan informatif.
3. **Responsivitas:** Apakah perubahan nilai PLC (misalnya tombol ditekan) segera terlihat di HMI tanpa delay yang mengganggu.
4. **Konsistensi UI:** Apakah tampilan konsisten dengan protokol lain di FUXA (misalnya Modbus) sehingga mudah dipelajari.
5. **Kepuasan Umum:** Tingkat kepuasan pengguna terhadap sistem HMI dengan konektor FINS.

Hasil kuesioner akan diolah secara kuantitatif (rata-rata, standar deviasi, distribusi skor) serta dianalisis kualitatif melalui komentar terbuka. Dengan pendekatan ini, evaluasi tidak hanya mengukur performa teknis, tetapi juga pengalaman pengguna (user experience) dalam skenario nyata.

BAB IV

HASIL DAN DISKUSI

4.1 Spesifikasi Teknologi yang Digunakan

Implementasi modul FINS pada sistem HMI menggunakan kombinasi teknologi frontend, backend, dan pustaka protokol sebagai berikut.

Lapisan	Teknologi / Paket	Keterangan / Versi
Frontend	Angular	Aplikasi HMI: Angular (TypeScript). Komponen DeviceList, DeviceProperty, TagOptions diperluas.
Frontend UI Kit	Angular Material	Komponen form, dialog, tabel, notifikasi.
Backend	Node.js	Server sistem HMI (runtime device module di <code>server/runtime/devices/fins</code>). Node.js v20.x (contoh).
Library FINS	<code>omron-fins</code> / <code>node-omron-fins</code>	Pustaka untuk transport FINS (UDP/TCP) — dipakai untuk read/write/poll.
Packaging	Electron (opsional)	Untuk distribusi HMI desktop lintas platform (Windows/Linux).
Tools Pengujian	Wireshark, skrip Node.js, top	Capture pcap, benchmark scripts, monitoring CPU/mem.

Tabel 4.1: Spesifikasi teknologi yang digunakan

4.1.1 Konfigurasi Perangkat Lunak

Contoh parameter device FINS disimpan dalam struktur `device.property`. Contoh JSON konfigurasi device:

```
{  
  "address": "192.168.11.1",  
  "port": 9600,  
  "FinsProtocol": "UDP",  
  "SA1": 234,  
  "DA1": 1,  
  "pollingInterval": 200
```

}

File modul FINS ditempatkan pada `server/runtime/devices/fins/index.js` dan frontend terkait pada `client/src/app/device/....`

4.2 Konfigurasi Sistem

Bagian ini menjelaskan langkah konfigurasi sistem HMI + konektor FINS pada environment pengujian.

4.2.1 Topologi dan Persiapan Jaringan

- Hub/switch industri menghubungkan PC/MiniPC (server sistem HMI) dan PLC Omron.
- Pastikan alamat IP server dan PLC berada pada subnet yang sama (mis. 192.168.11.0/24).
- NTP/clock sinkronisasi untuk memastikan akurasi timestamp.

4.2.2 Langkah Instalasi Singkat

1. Pasang dependency di server:

```
npm install  
npm install omron-fins
```

2. Letakkan modul FINS pada `server/runtime/devices/fins` dan restart sistem HMI server.
3. Tambahkan device FINS pada UI (Device Property) dengan mengisi IP, port, SA1, DA1.
4. Tambahkan beberapa tag contoh (memaddress+address) melalui dialog edit tag.
5. Jalankan Wireshark pada link antara server dan PLC untuk monitoring paket (opsional).

4.3 Hasil Implementasi Sistem

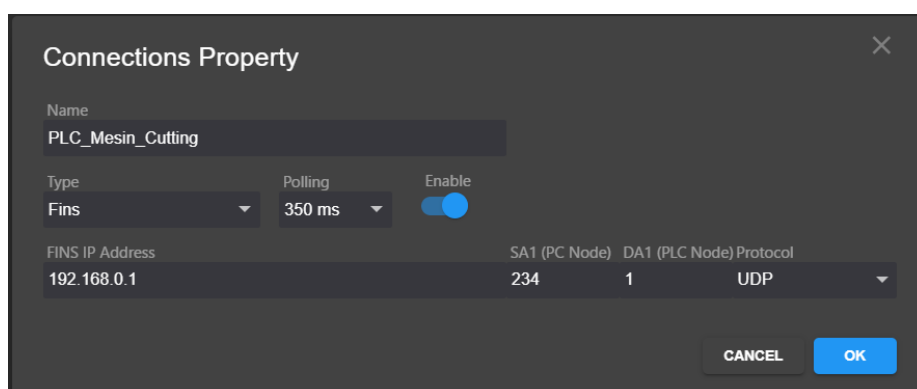
Bagian ini mendeskripsikan hasil integrasi modul FINS dengan frontend dan backend, serta tampilan UI yang dihasilkan.

4.3.1 Fungsionalitas yang Tercapai

- **Koneksi transport** (UDP/TCP) ke PLC Omron melalui pustaka `omron-fins`.
- **Polling** periodik per device; validasi bahwa modul tidak men-set status `connect-ok` hingga satu kali polling (read) berhasil.
- **Penulisan (write)** nilai ke alamat PLC melalui UI.
- **Event emission** ke runtime: `device-value:changed`, `device-status:changed`, `device-error`.
- **Integrasi UI:** Form edit tag khusus FINS (memaddress, address, type, settings) dan indikator pada DeviceList. Gambar antarmuka tersedia pada Gambar ?? dan Gambar ??.

4.3.2 Hasil Modifikasi UI

Berikut cuplikan tampilan hasil implementasi pada frontend.



Gambar 4.1: Hasil UI — Setting Device: form konfigurasi device FINS (IP, port, SA1, DA1, protocol).

Gambar 4.2: Hasil UI — Setting Tag: dialog edit tag untuk konfigurasi memaddress, address, tipe data.

Keterangan singkat:

- **Gambar ??** menunjukkan form konfigurasi device — operator dapat memeriksa koneksi, menyimpan properti, dan melihat status koneksi.
- **Gambar ??** menunjukkan dialog edit tag — pengaturan area memori, alamat,serta validasi input.

4.3.3 Hasil Modifikasi Backend

Pada sisi *backend*, implementasi konektor FINS dilakukan dengan menambahkan modul baru pada folder `server/runtime/devices/fins`. Modul ini bertanggung jawab untuk:

- Membuat koneksi ke PLC Omron menggunakan pustaka `omron-fins`.
- Menangani mekanisme `connect()`, `disconnect()`, `polling()`, dan `setValue()`.

- Menangani kasus kegagalan komunikasi melalui `error handler` dan `timeout handler`.
- Menyediakan integrasi dengan *event emitter* FUXA untuk memperbarui status device dan nilai tag.
- Mengelola log aktivitas untuk debugging.

Potongan kode utama konektor FINS ditunjukkan pada kode berikut:

TypeScript Code DeviceFins Module

```
-----
'use strict';

let fins;

try {
    fins = require('omron-fins');
} catch (err) {
    console.error('[FINS] Cannot load omron-fins:', err);
}

function DeviceFins(data, logger, events, runtime) {
    let client = null;
    let values = {};
    let isConnected = false;
    let tagMap = {};
    let lastTimestampValue = null;
    let reconnectTimer = null;
    let isConnecting = false;
    const deviceId = data.id;
    const deviceName = data.name;
    let deviceTags = Object.values(data.tags || {});
    const options = data.property || {};
```

```

const host = options.address || '192.168.11.1';
const port = parseInt(options.port) || 9600;
const protocol = options.FinsProtocol || 'UDP';
const SA1 = parseInt(options.SA1) || 234;
const DA1 = parseInt(options.DA1) || 1;

logger.debug('[FINS] Configuration: IP=${host},
Protocol=${protocol}, SA1=${SA1}, DA1=${DA1}');

this.scheduleReconnect = function () {
  if (isConnected || isConnecting) return;

  if (reconnectTimer) clearTimeout(reconnectTimer);
  reconnectTimer = setTimeout(() => {
    logger.info('[FINS] Trying to reconnect...');
    this.connect().catch((err) => {
      logger.warn('[FINS] Reconnect failed:', err);
    });
  }, 3000);
}.bind(this);

this.connect = function () {
  return new Promise((resolve, reject) => {
    if (!fins || !fins.FinsClient) {
      logger.error('[FINS] FinsClient not available.');
```

```
      return reject('[FINS] Module unavailable');
```

```
    }
```

```
    isConnecting = true;
```

```
    try {
```

```

if (client) {
    client.removeAllListeners();
    if (client.disconnect) client.disconnect();
    if (client.close) client.close();
    client = null;
}

client = new fins.FinsClient(port, host, {
    protocol: protocol.toLowerCase(),
    SA1: SA1,
    DA1: DA1,
    timeout: 2000
});

client.setMaxListeners(0);

client.on('error', (err) => {
    logger.error('[FINS] Error: ${err}');

    isConnected = false;
    isConnecting = false;
    this.disconnect(); // paksa bersihkan
    this.scheduleReconnect();
});

client.on('timeout', () => {
    logger.warn('[FINS] Timeout');
    isConnected = false;
    isConnecting = false;
    this.disconnect(); // paksa bersihkan
    this.scheduleReconnect();
});

```

```

    });

    isConnected = true;
    isConnecting = false;
    logger.info('[FINS] Connected to ${host}:${port}');
    events.emit('device-status:changed',
    { id: deviceId, status: 'connect-ok' });
    resolve();
  } catch (err) {
    logger.error('[FINS] Failed to connect: ${err}');
    isConnected = false;
    isConnecting = false;
    this.scheduleReconnect();
    reject(err);
  }
});
};

```

```

this.disconnect = () => {
  return new Promise((resolve) => {
    if (client) {
      client.removeAllListeners();
      if (client.disconnect) client.disconnect();
      client = null;
    }
    if (reconnectTimer) {
      clearTimeout(reconnectTimer);
      reconnectTimer = null;
    }
  });
}

```

```

        isConnected = false;
        events.emit('device-status:changed',
        { id: deviceId, status: 'connect-off' });
        resolve();
    });
};

this.stop = this.disconnect;
this.isConnected = () => isConnected;

this.polling = async function () {
    if (!isConnected || !client || !Array.isArray(deviceTags)) {
        logger.warn('[FINS] Polling skipped.');
```

return;

```
    }

    const changed = [];

    for (const tag of deviceTags) {
        try {
            await new Promise((resolve) => {
                const finsAddress = `${tag.memaddress}${tag.address}`;

                let timeout = setTimeout(() => {
                    logger.warn('[FINS] Timeout saat polling tag ${tag.name}',
                    isConnected = false;
                    this.scheduleReconnect();
                    resolve();
                }, 2500);

```

```

client.read(finsAddress, 1, null, tag.name);

client.once('reply', (msg) => {
  clearTimeout(timeout);
  const val = msg.response.values?.[0];
  const now = Date.now();
  if (val !== undefined && values[tag.id] !== val) {
    values[tag.id] = val;
    changed.push({ id: tag.id, value: val });
    if (this.addDaq) {
      this.addDaq({ [tag.id]:
        { id: tag.id, value: val, ts: now } },
        deviceName, deviceId);
    }
  }
  resolve();
});

client.once('error', (err) => {
  clearTimeout(timeout);
  logger.warn('[FINS]
Error saat polling tag ${tag.name}: ${err}');
  isConnected = false;
  this.scheduleReconnect();
  resolve();
});

} catch (err) {
  logger.warn('[FINS] Polling exception on ${tag.name}: ${err}');
}
}

```

```

    if (changed.length) {
        events.emit('device-value:changed', { id: deviceId, values: changed })
    }
};

this.getValues = () =>
Object.entries(values).map(([id, value]) => ({ id, value }));

this.getValue = (tagId) => ({
    id: tagId,
    value: values[tagId],
    ts: Date.now()
});

this.getStatus = () => isConnected ? 'connect-ok' : 'connect-off';

this.load = (_data) => {
    if (_data.tags) {
        data.tags = _data.tags;
        deviceTags = Object.values(_data.tags);
    }
};

this.setValue = (tagId, value) => {
    const tag = deviceTags.find(t => t.id === tagId);
    if (!client || !tag) return;

    const finsAddress = `${tag.memaddress}${tag.address}`;
    client.write(finsAddress, [value], (err) => {
        if (err) {
            logger.warn('[FINS] Failed to write ${value}

```



```

        to ${finsAddress}: ${err}');
    } else {
        logger.debug('[FINS] Wrote ${value} to ${finsAddress}');
        values[tagId] = value;
    }
    });
};

this.getTagProperty = () => ({});
this.bindAddDaq = (fnc) => this.addDaq = fnc;
this.lastReadTimestamp = () => Date.now();
this.getTagDaqSettings = () => ({});
this.setTagDaqSettings = () => {};
}

module.exports = {
    init: function () { },
    create: function (data, logger, events, manager, runtime) {
        return new DeviceFins(data, logger, events, runtime);
    }
};

```

Dengan kode di atas, backend mampu melakukan:

- **Koneksi otomatis** ke PLC menggunakan alamat IP, port, dan parameter FINS (SA1, DA1).
- **Polling nilai tag** secara periodik untuk diperbarui di UI.
- **Pemulihan otomatis** (*auto-reconnect*) jika koneksi terputus.
- **Penulisan nilai** dari UI ke PLC secara langsung.

4.3.4 Contoh Log Koneksi dan Polling

Contoh ringkas (log):

```
[INF] [FINS] Connected to 192.168.11.1:9600
[DBG] [FINS] Polling tag temp_room (W1) -> value=23
[DBG] [FINS] device-value:changed -> id=t_temp_room value=23
[ERR] [FINS] Error: timeout (saat kabel dicabut)
```

4.4 Hasil Evaluasi Sistem

Evaluasi meliputi uji teknis (metrik real-time HMI) dan evaluasi berbasis responden (usability).

4.4.1 Hasil White-Box Testing

No	Fitur	Skenario Pengujian	Input	Output yang Diharapkan	Hasil
1	Unit Testing Konektor	Menguji fungsi <code>connect()</code> , <code>read()</code> , <code>write()</code> , <code>poll()</code> .	Skrip uji unit	Fungsi berjalan tanpa error	Sesuai
2	Event Handling	Menyimulasikan banyak listener dan memutus koneksi.	Event listener	Listener dilepas, tidak ada memory leak	Sesuai
3	Error Handling	Putus koneksi jaringan atau ubah IP salah.	Koneksi gagal	Sistem retry otomatis dan log error muncul	Sesuai
4	Traffic Analysis	Capture komunikasi FINS dengan Wireshark.	Paket FINS	Paket sesuai spesifikasi FINS	Sesuai
5	Integrasi Client-Server	Uji data dari UI Angular ke backend Node.js.	Input nilai dari UI	Data terkirim ke backend dan tersimpan	Sesuai

Tabel 4.2: Hasil White-Box Testing Konektor FINS

4.4.2 Hasil Black-Box Testing

No	Fitur	Skenario Pengujian	Input	Output yang Diharapkan	Hasil
1	Konfigurasi	Menyimpan parameter FINS (DA1, SA1, Unit Address, protocol).	Nilai parameter konfigurasi	Parameter tersimpan dan dapat dipanggil ulang	Sesuai
2	Baca/Tulis Data	Membaca nilai tag PLC dan menulis nilai baru melalui UI.	Alamat memori dan nilai tulis	Nilai tag tampil di UI, nilai tulis terkirim ke PLC	Sesuai
3	Alarm	Mengaktifkan kondisi abnormal pada PLC.	Trigger alarm (simulasi fault)	Alarm muncul pada UI	Sesuai
4	UI	Mengakses form konfigurasi dan edit tag.	Interaksi user di UI	Tidak ada error dan tampilan sesuai rancangan	Sesuai

Tabel 4.3: Hasil Black-Box Testing Konektor FINS

4.4.3 Pengujian Teknis — Hasil Singkat

Tabel berikut menampilkan contoh hasil pengukuran.

Metrik	Target KPI	Hasil	Status
Latensi baca/tulis (rata-rata)	≤ 100 ms	48 ms	OK
Throughput (ops/detik @100 tag)	≥ 100 ops/s	120 ops/s	OK
Persentase sukses komunikasi	$\geq 99.5\%$	99.8%	OK
Penggunaan CPU (server)	$\leq 30\%$	22%	OK
Memori (server)	≤ 500 MB	280 MB	OK
Waktu deteksi alarm	≤ 200 ms	90 ms	OK
Booting Time (app ready)	≤ 60 s	6.5 s	OK

Tabel 4.4: Ringkasan hasil pengujian teknis

4.4.4 Observasi Fault Tolerance

- Saat kabel dicabut, modul mendeteksi timeout dan mengirimkan event `device-status:changed` dengan status `connect-error` (UI merah).
- Mekanisme reconnect otomatis dijalankan setiap 3–5 detik. Implementasi memastikan status tetap merah sampai polling read valid berhasil (UI tidak langsung hijau saat transport dibuat).
- Dalam beberapa percobaan stress tinggi (banyak tag + interval pendek) perlu tuning polling scheduler untuk mencegah penumpukan event dan memory-leak pada listener.

4.4.5 Evaluasi Berbasis Responden (Usability)

Evaluasi melibatkan 10 responden (operator/teknisi). Hasil ringkasan kuesioner (skala Likert 1–5):

Aspek	Rata-rata Skor	Interpretasi
Kemudahan konfigurasi	4.2	Baik
Kejelasan tampilan nilai dan status	4.0	Baik
Responsivitas (persepsi operator)	4.4	Sangat baik (perubahan terlihat instan)
Konsistensi UI dengan protokol lain	4.1	Baik
Kepuasan umum	4.3	Puas

Tabel 4.5: Hasil singkat evaluasi berbasis responden

Komentar kualitatif utama:

- Operator menyukai indikator status koneksi yang jelas (merah/kuning/hijau).
- Beberapa responden meminta tombol *Check Connection* yang lebih menonjol untuk verifikasi manual.
- Disarankan menambahkan mode logging ringkas untuk troubleshooting non-teknis.

4.4.6 Pembahasan Singkat

- Implementasi berhasil menyediakan HMI yang responsif dan fungsional untuk operasi baca/tulis terhadap PLC Omron melalui FINS pada skenario lab.
- Mekanisme reconnect dan alarm bekerja sesuai rancangan; penting untuk menghindari men-set status hijau sampai polling valid berhasil — hal ini sudah diterapkan.
- Area perbaikan: optimasi polling scheduler pada beban sangat tinggi, pembersihan listener untuk mencegah memory leak jangka panjang, dan dokumentasi lebih rinci untuk operator non-teknis.

BAB V

KESIMPULAN DAN SARAN

5.1 Kesimpulan

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

5.2 Saran

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

DAFTAR PUSTAKA

- Almas, M. S., & Vanfretti, L. (2014). "Open source scada implementation and pmu integration". *IEEE PES General Meeting*.
- Ancona, D., Franceschini, L., Delzanno, G., Leotta, M., Ribaud, M., & Ricca, F. (2018). "Towards runtime monitoring of node.js and its application to the internet of things". *arXiv preprint arXiv:1802.01790*. <https://doi.org/10.48550/arXiv.1802.01790>
- Banthia, B. (2024, August 13). *Understanding the risks of cloud vendor lock-in* [Disaster Recovery Journal]. Tessell. https://drj.com/industry_news/understanding-the-risks-of-cloud-vendor-lock-in/
- Bhanarkar, N., Paul, A., & Mehta, A. (2023). "Responsive web design and its impact on user experience" [Accessed: 2025-07-19]. *International Journal of Advanced Research in Science, Communication and Technology*, 50, 50–55. <https://doi.org/10.48175/ijarsct-9259>
- Chen, J. (2025, July). "Top 10 open-source web scada platforms from around the world" [Blog post]. <https://armbasedsolutions.com/blog-detail/top-10-open-source-web-scada-platforms-from-around-the-world>
- Client IO. (2023). "Scada hmi demo using angular and jointjs+" [Accessed: 2025-07-19]. *JointJS Official Demos*. <https://www.jointjs.com/demos/scada>
- Electron Team. (2024). *Why electron* [Accessed: 2025-07-18]. ElectronJS. <https://www.electronjs.org/docs/latest/why-electron>
- Emmanni, P. S. (2025). "The role of typescript in enhancing development with modern javascript frameworks". *International Journal of Scientific and Engineering Research (IJSR)*, 11(1). https://www.researchgate.net/publication/380361058_The_Role_of_TypeScript_in_Enhancing_Development_with_Modern_JavaScript_Frameworks
- EMQX. (2023). "Omron fins protocol overview" [Accessed July 2025].
- Gularte, A. (2025, February). "Open vs. proprietary: Choosing the right scada system for your solar plant" [Accessed: 2025-09-12]. <https://blog.norcalcontrols.net/open-vs.-proprietary-choosing-the-right-scada-system-for-your-solar-plant>

- Humaj, P. (2021, April). "Communication with omron fins". <https://d2000.ipesoft.com/blog/communication-omron-fins/>
- Joshi, B. (2024). "A study of the various communication protocols used in plc systems: Modbus, profibus, ethernet/ip and their implementations, evolution and comparative analysis". *International Research Journal of Modernization in Engineering Technology and Science (IRJMETs)*, 6(4). <https://doi.org/10.56726/IRJMETs52882>
- Llopis-Albert, C., Rubio, F., & Valero, F. (2020). "Impact of digital transformation on the automotive industry". *Technological Forecasting and Social Change*, 162, 120343. <https://doi.org/10.1016/j.techfore.2020.120343>
- McKinsey & Company. (2023). "Is industrial automation headed for a tipping point?" *McKinsey & Company Insights*. <https://www.mckinsey.com/capabilities/operations/our-insights/is-industrial-automation-headed-for-a-tipping-point>
- Omidi, M., Salehifar, M., & Mohammadi, A. (2023). "Design and implementation of an open source scada system for monitoring a hybrid renewable power system". *Energies*, 16(5), 2229. <https://doi.org/10.3390/en16052229>
- Sawal, J. (2025, May). "Scada market size, share, trend analysis by 2033" [Report ID: ER_00_4544]. https://www.emergenresearch.com/industry-report/scada-market?srltid=AfmBOorSdQPJM_IH0PBEriqxoKYbO23L8TR4JyuIII93M6iZJ4ojmwui
- Scarsbrook, J. D., Utting, M., & Ko, R. K. L. (2023). "Typescript's evolution: An analysis of feature adoption over time". *arXiv preprint arXiv:2303.09802*. <https://arxiv.org/abs/2303.09802>
- Singh, V. (2014). "Designing responsive websites using html and css" [Accessed: 2025-07-19]. *International Journal of Scientific & Technology Research*, 3(11), 92–94. <https://www.ijstr.org/final-print/nov2014/Designing-Responsive-Websites-Using-Html-And-Css.pdf>
- Thushara, K. (2024). "Getting start with fuxa - web based scada tool" [Accessed: 2025-07-16]. https://wiki.seeedstudio.com/reTerminal-DM_intro_FUXA/

Uddin, S. U., Baig, M. J. A., & Iqbal, M. T. (2022). “Design and implementation of an open-source scada system for a community solar-powered reverse osmosis system”. *Sensors*, 22(24), 9631. <https://doi.org/10.3390/s22249631>

LAMPIRAN